Chapter 8
Compression Engines

Abstract In this chapter we present libraries and APIs for lossless compression. Even with the geometric rise in memory capacity, the information theoretic compression gains provided at relatively cheap computing power can result in impressive performance and capacity gains. In this chapter we present with the help of examples, the use of the ZLIB and BZIP2 libraries. Examples are presented using C/C++ API as well as Python modules. More recently, LZMA and XZ Utilities also provide even better compression, albeit at slightly slower speed. Their use is also described in this chapter.

Contents

8.1 ZLIB Compression Library 156
8.2 LIBBZ2 and BZIP2 162
8.3 LZMA and XZ Utils 163
8.4 Conclusion 164

Although network speeds and memory capacities have expanded at a rapid rate in recent years, application demands for ever more features and capabilities have continued to place a constraint on the total memory space consumed by the program, or data which is communicated between cooperating processes. One solution to alleviating this concern is the use of compression, both lossless (for applications which need bit-equivalent data on both ends of the communication) or lossy (such as image compression). In the next section we discuss techniques for lossless compression including the use of *deflate*, and its implementation in ZLIB. As the performance gap between CPU and memories continue to expand, spending CPU cycles for compression and decompression, which reduces the memory requirement is indeed beneficial.

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_8, © Springer Science+Business Media, LLC 2011

8.1 ZLIB Compression Library

`zlib` is a loss-less compression and decompression library. It has the advantage of input independent memory requirements, as well as the compression algorithm produces excellent compression of data. The `zlib` library is an implementation of the `deflate` algorithm which in turn uses Huffman coding to perform compression. The `zlib` library can be used as a buffer compression library (which in turn can be used to compress files), or it can be used to read files compressed using the `gzip` tool. The `zlib` functions `gzread` allows reading compressed files. See Section 8.1.2 for more details on the file functions of `zlib`.

The compressed data format used by the in-memory functions is the zlib format (by default), which is a zlib wrapper (RFC 1950), around a deflate stream (RFC 1951). A Deflate stream consists of a series of blocks. Each block is preceded by a 3-bit header:

1. Bit 0 :marker,
   - 1: last block in stream,
   - 0: more blocks.
2. Bit 1 and 2: represent the encoding method:
   - 00: a stored/raw/literal section follows, between 0 and 65,535 bytes in length,
   - 01: static Huffman compressed block (pre-agreed tree),
   - 10: dynamic Huffman block alongwith tree,
   - 11: reserved.

Most data blocks are compressed using the dynamic Huffman coding, where the tree is contained after the header. The two techniques used for data compression in DEFLATE are (i) duplicate string referencing and (ii) bit-reduction by symbol encoding. Duplicate strings are *back referenced* (upto a maximum distance of 32K bytes), and symbol encoding is performed using the Huffman encoding method. The lookback for the string matching contributes significantly to the quality and performance of the compression library. In this section we discuss the reference `zlib` implementation since it is widely used in many open-source software, as well as being part of Internet standard (RFC 1950) and the Java JAR format.

The `zlib` functionality is accessed using an opaque ZSTREAM object as shown in Listing 8.1. The declaration of this stream structure can be found in `zlib.h` and is shown below:

```
typedef struct z_stream_s {
  Bytef    *next_in;  /* next input byte */
  uInt     avail_in;  /* no. bytes available next_in */
  uLong    total_in;  /* input bytes read so far */

  Bytef    *next_out; /* location of next output */
  uInt     avail_out; /* free space at next_out */
  uLong    total_out; /* no, bytes output so far */

  char     *msg; /* error message, NULL if no error */
  struct internal_state FAR *state; /* opaque */
```

```
  alloc_func zalloc;  /* internal allocator */
  free_func  zfree;   /* internal deallocator */
  voidpf     opaque;  /* pvt. data to allocator */

  int     data_type;  /* binary or text */
  uLong   adler;      /* adler32 value */
  uLong   reserved;   /* reserved  */
} z_stream;
```

During compression or decompression, `next_in` and `avail_in` must be updated when `avail_in` becomes zero. Similarly, the members `next_out` and `avail_out` have to be updated. The memory allocation and `opaque` member must be initialized. When compressing data-buffers, compression can be performed in a single step if the input/output buffers have appropriate memory allocated for them. A comprehensive example of using ZLIB with XDR is given in Listing 8.1, which uses the single-shot method of compression. Otherwise, the stream object's `avail_out` member has to be monitored to ensure that space is available for output. The fields `total_in` and `total_out` are used for statistics or progress reports and contain the number of input, and output bytes, read and written, respectively. The main functions provided by the `zlib` library are shown in Table 8.1.

Table 8.1 Major functions of the zlib library

`zlib` function	Description
`zlib_version`	return version number
`deflateInit`	initialize the stream state for compression
`deflateInit2`	same as above, but has more options
`deflate`	compress as much data as possible
`deflateEnd`	free dynamically allocated structures
`inflateInit`	initialize stream for decompression
`inflateInit2`	same as above, but has more options
`inflate`	decompress as much data as possible
`inflateEnd`	free dynamically allocated structures
`compress`	utility function to compress buffer
`compress2`	same as above, but has `level`
`compressBound`	function to return upper bound on length of output
`uncompress`	utility function to perform decompression
`adler32`	update running Adler-32 checksum
`crc32`	update running CRC32

The functions `deflateInit2` has customizable options including (i) compression method, (ii) windowBits, (iii) memory level, and (iv) compression strategy. Similarly, the `inflateInit2` has only one extra parameter, the number of window bits (specified as a base two logarithm), which controls the maximum window size.

```
// \file rdb.cpp
// \author Sandeep Koranne, (C) 2010
```

```
// \description Use of zlib, bzip2, LZMA with XDR
#include <iostream>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <rpc/types.h>
#include <rpc/xdr.h>
#include <boost/crc.hpp>     // actual CRC
#include <zlib.h>

static size_t MAX_LEN = 0;
static char *BUFFER = NULL;
static char *ZLIB_BUFFER = NULL;
static unsigned long CHECKSUM = 0;
// Note: use of CRC32 is not endian-proof as we always
// across machines, but is acceptable as a test case
// against data corruption.

int WriteNumberStream( int N, char** buf ) {
  boost::crc_32_type boost_crc;
  XDR xdrs;
  if( MAX_LEN <= ( N * sizeof( int ) ) ) {
    *buf = new char[ ( N * sizeof( int ) ) ];
    MAX_LEN = N * sizeof( int );
  }
  xdrmem_create( &xdrs, *buf, MAX_LEN, XDR_ENCODE );
  for( int i=0; i < N; ++i ) {
    int x = ( rand() % 100 ) ^ (i) ;
    boost_crc.process_byte( x & 0x0FF0 ); // middle order bits
    if (!xdr_int( &xdrs, &x ) ) return -1;
  }
  int written_len = xdr_getpos( &xdrs );
  //std::cout<<written_len<<"\t"<<boost_crc.checksum()<<"\t";
  //std::cout << "Wrote " << written_len << " bytes.\n";
  //std::cout << "CRC32 checksum = " << boost_crc.checksum();
  CHECKSUM = boost_crc.checksum();
  return written_len;
}

int ReadNumberStream( int N, char* buf ) {
  boost::crc_32_type boost_crc;
  XDR xdrs;
  xdrmem_create( &xdrs, buf, N, XDR_DECODE );
  int x;
  int count=0;
  while( true ) {
    if( !xdr_int( &xdrs, &x ) ) break;
    boost_crc.process_byte( x & 0x0FF0 );
    count++;
  }
  if( CHECKSUM != boost_crc.checksum() ) {
    std::cerr << "CHECKSUM failed.\n";
    exit( 1 );
  }
  //std::cout << "Read " << count << " integers.\n";
  //std::cout << "CRC32 checksum = " << boost_crc.checksum();
  return count;
}

int CompressXDRStream( char *input, int len, char* output, int MAX_LEN ) {
  z_stream zstrm;
  int flush = Z_NO_FLUSH;
  int compressed_till_now = 0;
  zstrm.zalloc = Z_NULL;
  zstrm.zfree = Z_NULL;
  zstrm.opaque = Z_NULL;
  int ret = deflateInit( &zstrm, 3 ); // level = 3
```

```
70    if( ret != Z_OK ) {
        std::cerr << "ZLIB stream construction failed.\n";
        exit( 1 );
      }
      zstrm.avail_in = len;
75    zstrm.next_in  = (unsigned char*)input;
      zstrm.avail_out = MAX_LEN;
      zstrm.next_out = (unsigned char*)output;
      do {
        // we expect the whole buffer in single shot
80      ret = deflate( &zstrm, flush );
        if( ret != Z_OK ) {
          std::cerr << "ZLIB failure\n";
          exit( 1 );
        }
85    } while( zstrm.avail_in > 0 );
      ret = deflate( &zstrm, Z_FINISH );
      compressed_till_now = MAX_LEN - zstrm.avail_out;
      //std::cout<<"Wrote "<<compressed_till_now<<" bytes during deflate.\n";
      deflateEnd( &zstrm );
90    return compressed_till_now;
    }

    int DecompressXDRStream( char *input, int len, char* output, int MAX_LEN ) {
      z_stream zstrm;
95    int flush = Z_NO_FLUSH;
      int decompressed_till_now = 0;
      zstrm.zalloc = Z_NULL;
      zstrm.zfree = Z_NULL;
      zstrm.opaque = Z_NULL;
100   int ret = inflateInit( &zstrm );
      if( ret != Z_OK ) {
        std::cerr << "ZLIB stream construction failed.\n";
        exit( 1 );
      }
105   zstrm.avail_in = len;
      zstrm.next_in  = (unsigned char*)input;
      zstrm.avail_out = MAX_LEN;
      zstrm.next_out = (unsigned char*)output;
      do {
110     // we expect the whole buffer in single shot
        ret = inflate( &zstrm, flush );
        if( ret == Z_STREAM_ERROR ) {
          std::cerr << "ZLIB failure\n";
          exit( 1 );
115     }
      } while( zstrm.avail_in > 0 );
      ret = deflate( &zstrm, Z_FINISH );
      decompressed_till_now = MAX_LEN - zstrm.avail_out;
      //std::cout<<"Read "<<decompressed_till_now<<" bytes during deflate.\n";
120   inflateEnd( &zstrm );
      return decompressed_till_now;
    }

    int main( int argc, char *argv[] ) {
125   int number_to_write = 1000;
      if( argc > 1 ) number_to_write = atoi( argv[1] );
      int numWritten = WriteNumberStream( number_to_write, &BUFFER );
      if( numWritten < 0 ) {
        std::cerr << "XDR stream overflowed, allocate more memory.\n";
130     exit( 1 );
      }
      if( MAX_LEN ) ZLIB_BUFFER = new char[ MAX_LEN * sizeof( int ) ];
      int comp_len = CompressXDRStream( BUFFER, numWritten,
                                        ZLIB_BUFFER, MAX_LEN );
135   std::cout << numWritten << "\t" << comp_len << "\n";
      memset( BUFFER, 0, MAX_LEN ); // initialize to 0
```

```
    int decomp_len = DecompressXDRStream( ZLIB_BUFFER, comp_len,
                                          BUFFER, MAX_LEN );
    int numRead = ReadNumberStream( numWritten, BUFFER );
140
    return 0;
}
```

Listing 8.1 Using ZLIB with XDR

Using the utility functions `uncompress` requires the transmission of the length of the uncompressed stream alongwith the compressed data as that is not stored in the compressed data. The `compress` function is declared as:

```
compress( Bytef *dest, uLongf *destLen,
          const Bytef *source, uLong sourceLen );
```

The size of the compressed stream is returned in `destLen`. Using the `compressBound` function an output buffer to accumulate the compressed data can be allocated prior to calling `compress`. The `uncompress` function is similarly declared:

```
uncompress( Bytef *dest, uLongf *destLen,
            const Bytef *source, uLong sourceLen );
```

The output buffer length has to be known to the application in advance. On return, `destLen` contains the number of bytes (which can be compared with the application data). Both these functions return `Z_OK` if the action was performed without error.

8.1.1 Compression ratio

This section is subjective, and depending on the data present in the stream compression ratio can vary significantly. We presented an example in Listing 8.1. Running this example on several different sizes of input yielded the plot as shown in Figure 8.1.

8.1.2 gzip file access functions

The `zlib` library supports reading and writing `gzip` (RFC 1952) files (also called '.gz' files). The main functions are shown in Table 8.2.

8.1.3 Integration of zlib and gzip in Python

In addition to using the library from your own C/C++ programs as we have shown in Listing 8.1, it is also possible to use `zlib` and `gzip` from Python. Consider the example shown below:

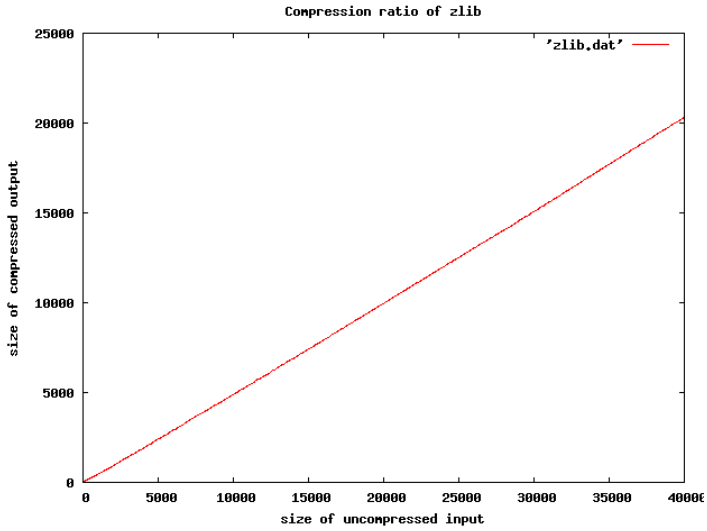


Fig. 8.1 Compression ratio for zlib

Table 8.2 Major `gzip` functions of the zlib library

`gzip` function	Description
`gzopen`	open filename for reading with mode,
`gzdopen`	open file descriptor for reading with mode
`gzbuffer`	set internal buffer size
`gzsetparams`	set level and strategy for file
`gzread`	read length (uncompressed) of bytes into buffer
`gzwrite`	write length (uncompressed) to file
`gzprintf`	format, convert and write to file
`gzputs`	write null-terminated string to file
`gzgets`	read null-terminated string of given length from file
`gzflush`	flush output to file
`gzseek`	sets starting position for next read/write
`gztell`	return position (uncompressed) of next read/write
`gzoffset`	return current offset of file
`gzeof`	return 1 if end-of-file indicator is set
`gzclose`	close the file

```
import sys
import gzip
def ParseFile( fileName ):
    print "Reading file %s" %fileName
    if poly_file_name.find( '.gz' ) != -1:
        file = gzip.open( fileName, 'rb' )
        lines = file.readlines()
        file.close()
    else:
```

```
        f = open( fileName, 'r' )
        lines = f.readlines()
        f.close()
    for l in lines:
        # computation on contents
```

As the above example shows using `zlib` in Python is simple, and transparent.

8.2 LIBBZ2 and BZIP2

The `bzip2` implements the Burrows-Wheeler block-sorting text compression algorithm. The underlying library of `bzip2` is `libbz2`. An advantage of using `bzip2` is that internally it organizes data as chunks of 900k, so partially damaged files can be recovered (using the `bzip2recover` program). The major functions of the `libbz2` library are shown in Table 8.3.

Table 8.3 Major functions of the libbz2 library

`zlib` function	Description
`BZ2_bzCompressInit`	prepares for compression
`BZ2_bzCompress`	compress the data, change state
`BZ2_bzCompressEnd`	cleanup data structures
`BZ2_bzDecompressInit`	prepare for decompression
`BZ2_bzDecompress`	decompress the data
`BZ2_bzDecompressEnd`	cleanup data structures
`BZ2_bzReadOpen`	prepare to read data from FILE*
`BZ2_bzRead`	read data into buffer
`BZ2_bzReadClose`	close FILE
`BZ2_bzWriteOpen`	open FILE for writing
`BZ2_bzWrite`	write compressed data to FILE
`BZ2_bzWriteClose`	close FILE
`BZ2_bzBuffToBuffCompress`	one-shot buffer compression
`BZ2_bzBuffToBuffDecompress`	one-shot buffer decompression

The stream for `libbz2` is shown below:

```
struct {
   char *next_in;              /* data to be compressed  */
   unsigned int avail_in;      /* length of input data remaining */
   unsigned int total_in_lo32;/* statistics */
   unsigned int total_in_hi32;
   char *next_out;             /* output buffer */
   unsigned int avail_out;     /* length of output buffer remaining */
   unsigned int total_out_lo32;/* statistics */
   unsigned int total_out_hi32;
   void *state;                /* internal state */
   void *(*bzalloc)(void *,int,int); /* memory allocator function */
```

```
   void (*bzfree)(void *,void *);     /* deallocator function */
   void *opaque;                      /* pointer given to allocator*/
} bz_stream;
```

8.2.1 Integration of bzip2 in Python

In addition to using the library from your own C/C++ programs it is also possible to use `bzip2` from Python. Consider the example shown below:

```
import bz2
def ParseFile( fileName ):
    print "Reading file %s" %fileName
    if poly_file_name.find( '.bz2' ) != -1:
        file = bz2.BZ2File( fileName, 'rb' )
        lines = file.readlines()
        file.close()
    else:
        f = open( fileName, 'r' )
        lines = f.readlines()
        f.close()
    for l in lines:
        # computation on contents
```

8.3 LZMA and XZ Utils

The Lempel-Ziv-Markov chain algorithm is implemented in the LZMA toolchain and library. The underlying library for the XZ tools is the `liblzma` library. We describe the compression and decompression functions of this library in this section.

The API for decoding LZMA compressed data is shown below:

```
SRes LzmaDecode(Byte *dest,
                SizeT *destLen,
                const Byte *src,
                SizeT *srcLen,
                const Byte *propData,
                unsigned propSize,
                ELzmaFinishMode finishMode,
                ELzmaStatus *status,
                ISzAlloc *alloc);
```

The memory allocators can be set to use the system wide `malloc` and `free` as:

```
void *SzAlloc(void *p, size_t size)
   { p = p; return malloc(size); }
void SzFree(void *p, void *address)
   { p = p; free(address); }
ISzAlloc alloc = { SzAlloc, SzFree };
```

Compressing data using LZMA is shown below:

```
  CLzmaEncHandle enc;
  enc = LzmaEnc_Create(&g_Alloc);
```

```
    if (enc == 0) return SZ_ERROR_MEM;
    LzmaEncProps_Init(&props);
5   res = LzmaEnc_SetProps(enc, &props);
    HRes LzmaEncode(Byte *dest,
                    SizeT *destLen,
                    const Byte *src,
                    SizeT srcLen,
10                  CLzmaEncProps *props,
                    Byte *propsEncoded,
                    SizeT *propsSize,
                    int writeEndMark,
                    ICompressProgress *progress,
15                  ISzAlloc *alloc,
                    ISzAlloc *allocBig);
```

The distinction between the allocators is done for systems which support large page size tables for efficient allocation of large memory buffers.

8.3.1 XZ Utils

An implementation of LZMA in the XZ utilities is available on GNU/Linux systems. The compression engines are also available with the GNU `tar` program using the `-J` command-line option (see Section 11 for a description of the command-line options). We experimented with compressing a file of 13 Mb using `gzip`, `bzip2` and `xz`, the results are shown in Table 8.4. It shows that XZ utilities can achieve significantly better compression, albeit at a higher runtime. As shown in Table 8.4 XZ (with the `-e`, extreme option) achieves almost 50% compression, while `gzip` and `bzip2` achieve (only) 0.84.

Table 8.4 Compression ratio of various tools

Tool name	Ratio	Comp. time (s)	Decomp. time (s)
`gzip -9`	0.84	02.59	0.46
`bzip2 -9`	0.84	11.99	3.96
`xz -e`	0.55	24.20	2.18

8.4 Conclusion

In this chapter we presented libraries and APIs for lossless compression. Using short examples, we presented the use of the ZLIB and BZIP2 libraries. Examples were presented using C/C++ API as well as Python modules. The use of LZMA and XZ Utilities was also discussed.